

UNIVERSIDAD DE GUADALAJARA



Centro Universitario de Ciencias Exactas e Ingenierías

Ingeniería en Computación.

Profesor: LUIS ANGEL BELTRAN CARRILLO

Materia: Fundamentos de Inteligencia Artificial

Equipo: Braulio Yael Carranza Zamora, Mariana González Velázquez,

Carlos Rivera Padilla jr, Vanessa Araceli Vázquez Mojica

Practica: 1-3 Agente Reactivo Simple

Fecha: 17 de septiembre de 2025

introducción:

Los algoritmos de búsqueda nos permiten explorar sistemáticamente un conjunto de posibles estados para encontrar una solución a un problema. Este reporte analiza dos de los algoritmos de búsqueda no informada más clásicos: la Búsqueda en Amplitud (BFS) y la Búsqueda en Profundidad (DFS).

Objetivo:

Implementar los algoritmos de búsqueda no informada en problemas de prueba para poder comparar su desempeño.

implementación:

Desarrollar un programa que encuentre la mejor solución a los problemas planteados, usando los algoritmos de búsqueda no informada vistos en clase (búsqueda en profundidad, búsqueda en amplitud, etc).

Se debe hacer el planteamiento de los problemas, en base a los conceptos: del espacio de estados las acciones (función sucesora), test objetivo y el costo del camino. Con base a esto, se definen los árboles y la estrategia de búsqueda.

Desarrollo:

BFS Aplicado al 8-Puzzle:

Garantía de la solución óptima: BFS siempre encuentra el camino más corto. En el contexto del puzzle, esto significa que la secuencia de movimientos hallada es la más breve posible para llegar a la solución.

Completo: Si el puzzle tiene una solución, BFS la encontrará sin falta.

Exploración exhaustiva por niveles: El algoritmo revisa todos los posibles tableros a 1 movimiento de distancia, luego todos los que están a 2 movimientos, y así sucesivamente. Esto evita que se pierda en secuencias de movimientos muy largas e ineficientes.

Alto consumo de memoria: Su principal desventaja es que necesita almacenar una gran cantidad de estados (posiciones del tablero) en la memoria. A medida que se aleja del estado inicial, el número de tableros a revisar en cada "capa" crece exponencialmente.

DFS Aplicado al Laberinto:

Rutas directas, pero no óptimas: DFS se adentra profundamente por un único camino, lo que puede resultar en rutas muy largas y enrevesadas, aunque eventualmente encuentre el destino. La solución hallada casi nunca es la más corta.

Bajo consumo de memoria: A diferencia de BFS, solo necesita guardar en memoria los nodos del camino que está explorando actualmente. Esto lo hace muy eficiente para laberintos grandes y complejos.

Rapidez variable (depende de la "suerte"): Puede ser muy rápido si la primera ruta que explora lo lleva directamente a la salida. Sin embargo, puede ser muy lento si se queda "atascado" explorando una sección grande y sin salida del laberinto.

Riesgo de no encontrar solución: Sin un límite de profundidad, DFS podría quedar atrapado en un ciclo o un camino infinito en laberintos más complejos. Por ello, la implementación usa un límite para evitarlo.

Problema 1: Solución del 8-Puzzle

¿Cómo resuelve el problema?

1. **Estado Inicial Aleatorio:** La función `generar_estado_inicial()` crea un tablero con las piezas desordenadas de forma aleatoria, cumpliendo con el requisito.
2. **Implementación de BFS:** La función `bfs(inicial)` explora el puzzle "nivel por nivel", garantizando que la primera solución que encuentre sea la que tiene el **menor número de movimientos** (la más óptima).
3. **Implementación de DFS:** La función `dfs(inicial)` explora una secuencia de movimientos hasta el final antes de retroceder. Encuentra una solución, pero no necesariamente la más corta. Incluye un límite de profundidad para evitar que se quede en bucles infinitos.

Problema 2: Búsqueda de Camino en un Plano 2D

¿Cómo resuelve el problema?

1. **Plano con Obstáculos:** La función `generar_plano(prob_obstaculo=0.25)` crea un plano complejo de 20x20 que sirve como laberinto. El problema se modela como una **cuadrícula** donde cada celda es una posición (x,y).
2. **Implementación de BFS:** La función `bfs_pasos(plano, inicio, destino)` encuentra el **camino más corto** desde el círculo rojo (inicio) hasta la estrella roja (destino). Su exploración expansiva asegura la optimalidad de la ruta.
3. **Implementación de DFS:** La función `dfs_pasos(plano, inicio, destino)` también encuentra un camino, pero sigue una ruta más serpenteante y casi nunca es la más corta.
4. **Visualización:** El código utiliza `matplotlib` para animar el proceso de búsqueda, mostrando cómo cada algoritmo explora el plano y finalmente dibuja la trayectoria encontrada.

Tabla comparativa:

Algoritmo	Ruta / Solución	Uso de Memoria	Rendimiento (Nodos Visitados)	Ideal Para:
BFS (Amplitud)	Óptima (Siempre la más corta)	Alto. Requiere almacenar todos los nodos de cada nivel.	Medio a Alto. Explora todos los caminos cortos antes de encontrar la solución.	Encontrar la solución con menos pasos cuando la memoria no es un problema.
DFS (Profundidad)	No óptima. Encuentra cualquier solución,	Bajo. Solo almacena el camino actual.	Bajo a Muy Alto. Puede tener suerte y encontrarla rápido, o	Encontrar cualquier solución rápidamente en problemas con

Algoritmo	Ruta / Solución	Uso de Memoria	Rendimiento (Nodos Visitados)	Ideal Para:
	usualmente una muy larga.		explorar ramas muy largas inútilmente.	muchos estados, ahorrando memoria.
IDDFS (Profundidad Iterativa)	Óptima (Garantiza la ruta más corta)	Bajo. Hereda el bajo consumo de memoria de DFS.	Alto. Es su principal desventaja. Vuelve a visitar los nodos de niveles anteriores en cada nueva iteración.	El mejor de los dos mundos: la optimalidad de BFS con la eficiencia de memoria de DFS. Es a menudo el mejor algoritmo de búsqueda no informada.

Conclusión:

En resumen, comprender las fortalezas y debilidades de estos algoritmos es esencial para diseñar soluciones de inteligencia artificial eficientes y efectivas.

Referencia:

Russell, S. J., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.